

UNITÉ 1 · SEMAINE 1

Introduction aux systèmes d'exploitation

📅 Semaine 1 · dates à confirmer </> Laboratoire TD0 (corrigé + défi IA) ~ 4 animations

MANUEL M1 · THÉORIE DE L'UNITÉ

Introduction aux systèmes d'exploitation

Lecture de base de l'unité 1 (≈ 1 h). Les chiffres de la machine du cours proviennent de l'exécution des commandes du TD0 (Ubuntu 26.04, Linux 7.0).

SOMMAIRE

- | | | | |
|----|--|----|---|
| §1 | Qu'est-ce qu'un système d'exploitation ? | §4 | Le démarrage |
| §2 | Couches et modes | §5 | Où ils sont aujourd'hui et comment ils y sont arrivés |
| §3 | Architectures de noyau | | |

§1 Qu'est-ce qu'un système d'exploitation ?

Un système d'exploitation est le logiciel qui se place entre les programmes et le matériel. Il a deux missions :

- **Gérer les ressources.** Il répartit le temps processeur entre les processus, alloue et protège la mémoire, ordonne l'accès aux disques, au réseau et à l'écran, et gère l'énergie. Il résout les conflits quand plusieurs programmes veulent la même chose en même temps.
- **Offrir une machine plus facile à utiliser.** Au lieu de secteurs de disque, d'interruptions et de registres de contrôleurs, les programmes voient des *abstractions* : processus, fichiers, sockets, mémoire virtuelle. Un programme qui écrit un fichier fonctionne de la même façon avec un SSD, un disque réseau ou une clé USB.

Il **protège** aussi : un programme ne peut ni lire la mémoire d'un autre programme ni contourner les permissions d'un fichier. Cette protection repose sur l'existence, dans le matériel, de différents modes d'exécution (§2).

§2 Couches et modes

De haut en bas : **applications** (bash, un navigateur, vos programmes) → **bibliothèques** (glibc, libssl) → **interface des appels système** → **sous-systèmes du noyau** (ordonnanceur, mémoire, VFS, réseau) → **pilotes** → **matériel**. Les deux premières couches s'exécutent en **mode utilisateur** ; les trois suivantes, en **mode noyau**. La frontière entre les deux ne se franchit qu'avec un appel système (unité 2).

Tout ce que fait une bibliothèque n'a pas besoin du noyau: `printf` accumule le texte dans un tampon et `malloc` distribue de la mémoire qu'il a déjà demandée. Ce n'est que lorsqu'il faut le matériel — écrire vraiment, demander plus de mémoire — que la bibliothèque fait un appel système.

Pratiquez → animation A1.1 · Les couches du système

§3 Architectures de noyau

CONCEPTION	CE QUI S'EXÉCUTE EN MODE NOYAU	POUR	CONTRE	EXE
Monolithique	Tout: ordonnanceur, mémoire, systèmes de fichiers, réseau, pilotes	Rapide: un appel, un franchissement à l'aller et un au retour	Un bogue dans un pilote peut faire tomber le système	Linux monolithique
Micronoyau	Le minimum: communication entre processus (IPC), ordonnancement, mémoire	Isolation: un pilote défaillant est relancé	Plus de franchissements et de messages par opération	selon le noyau
Hybride	La plupart des services; quelques sous-systèmes en mode utilisateur	Un compromis entre les deux	Hérite d'une partie des problèmes des deux	Windows, macOS

Linux est monolithique mais modulaire: la machine du cours a **258 modules chargés** (`lsmod`), des pilotes qu'on ajoute et retire sans redémarrer. Une quatrième idée, l'*unikernel*, compile l'application et le noyau en une seule image spécialisée, utile dans le cloud.

Pratiquez → animation A1.2 · Architectures de noyau

§4 Le démarrage

1. **Firmware UEFI**. Il teste le matériel, initialise la mémoire et les bus et cherche dans sa mémoire non volatile (NVRAM) quoi démarrer.
2. **Chargeur d'amorçage** (GRUB ou systemd-boot). Il charge en mémoire le noyau (`vmlinuz`) et un système de fichiers initial (`initramfs`) avec les pilotes nécessaires pour monter le disque.
3. **Noyau**. Il se décompresse, met en place la mémoire virtuelle et les processeurs, détecte les périphériques, monte la racine et lance le **premier processus, le PID 1**.

4. **Espace utilisateur.** `systemd` (PID 1) démarre les services en parallèle jusqu'à la connexion.

Tous les processus descendent de lui.

Sur la machine du cours, `systemd-analyze` a mesuré: firmware 9,1 s, chargeur 4,9 s, **noyau 2,6 s** et espace utilisateur 14,5 s (total 30,9 s). Le noyau est l'étape la plus courte.

Sortie réelle

```
$ uname -srm; grep PRETTY_NAME /etc/os-release; ps -p 1 -o comm=; lsmod | tail -n +2 |
wc -l; nproc; systemd-analyze
Linux 7.0.0-34-generic x86_64
PRETTY_NAME="Ubuntu 26.04 LTS"
systemd
237
28
Startup finished in 8.661s (firmware) + 3.568s (loader) + 3.253s (kernel) + 16.571s
(userspace) = 32.053s
graphical.target reached after 16.569s in userspace.
```

Pratiquez → animation A1.3 · Le démarrage, et TD0 · 5

§5 Où ils sont aujourd'hui et comment ils y sont arrivés

On trouve des systèmes d'exploitation dans les supercalculateurs (depuis novembre 2017, la liste TOP500 est entièrement dominée par Linux; le n° 1 de juin 2026, LineShine, utilise Kylin OS, une distribution basée sur Linux), dans les serveurs du cloud, dans les téléphones (Android utilise le noyau Linux; iOS, le noyau XNU), dans les voitures et les appareils médicaux (souvent avec des micronoyaux temps réel comme QNX) et dans des millions d'appareils embarqués.

ÉPOQUE	IDÉE CLÉ
Années 1950	Traitement par lots: une tâche à la fois, sans interaction.
Années 1960	Multiprogrammation et temps partagé (CTSS, Multics): plusieurs utilisateurs à la fois, le processeur est partagé.
1969–1973	Unix aux Bell Labs: processus, fichiers et tubes comme abstractions simples; réécrit en C.
Années 1980–1990	Ordinateurs personnels; Linux (1991) et Windows NT (1993).
2007– années 2010	Smartphones, virtualisation et conteneurs dans le cloud.
2025–2026	Systèmes d'exploitation qui accueillent des agents d'IA avec leur propre identité, leurs permissions et leur audit (unité 9).